

Universidad Autónoma de Baja California

Facultad de Ciencias de la Ingeniería y Tecnología



Estructuras de Datos

Programación Genética - Práctica Unidad 4

Integrantes:

- David Osvaldo Calderón Salgado
- Absalón Castellano Flores
- Miguel Ángel Lara Tarín
- Jared Alejandro López Aguilera

Docente: José Manuel Muñoz Contreras

Código de la Materia: 40007

28 de abril de 2026

Introducción

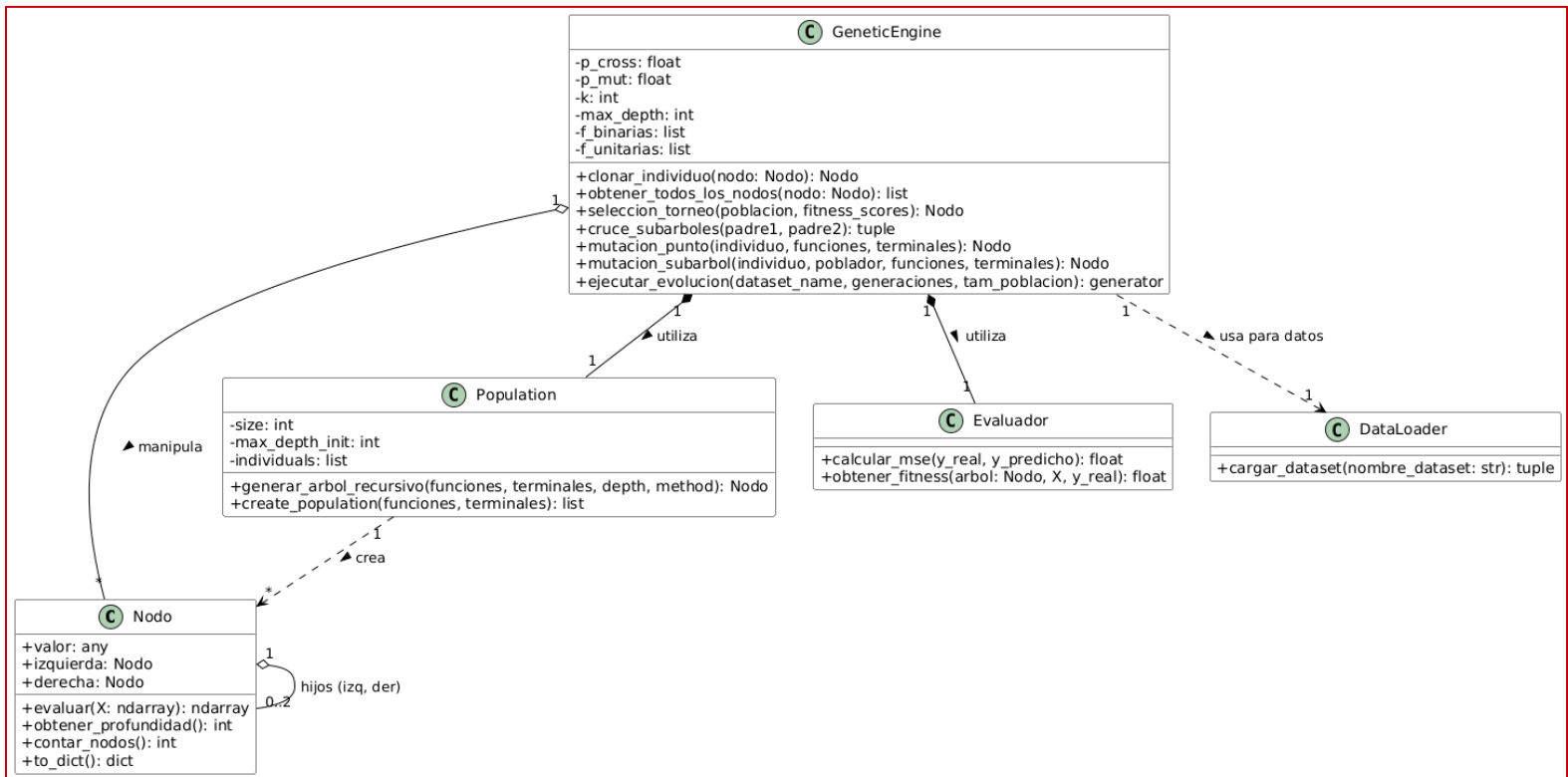
Este proyecto consiste en el desarrollo de un motor de Programación Genética (GP) construido desde cero mediante Python, utilizando únicamente la librería numpy para el procesamiento vectorial y pandas para la ingesta de datos. El objetivo principal es resolver problemas de Regresión Simbólica sobre seis conjuntos de datos clásicos (Concrete, Tower, Yacht, Housing, Energy Cooling y Energy Heating), descubriendo fórmulas matemáticas que minimicen el Error Cuadrático Medio (MSE) entre las predicciones del modelo y los valores reales.

1. Diseño de Software

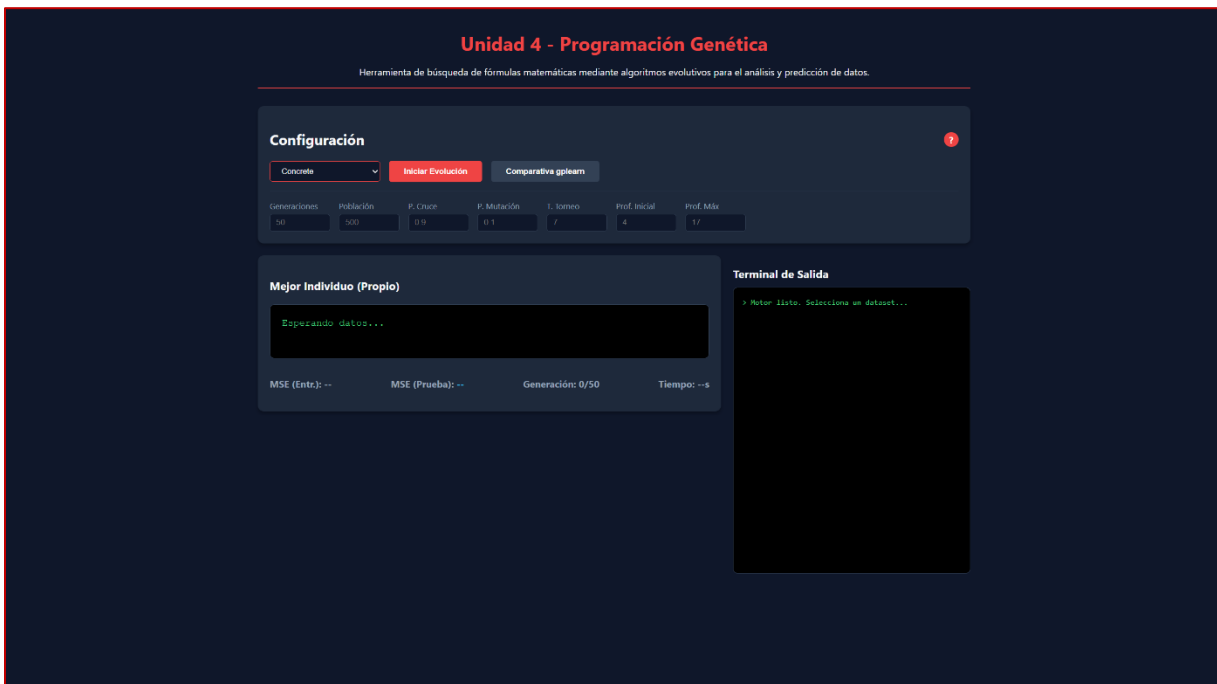
Para este sistema se diseñó una arquitectura orientada a objetos dividida en módulos independientes, garantizando así la escalabilidad y el cumplimiento de las restricciones de la práctica:

- **Clase Nodo:** Es la estructura base del árbol de sintaxis abstracta (AST). Contiene el valor (operador o terminal) y las referencias a sus hijos. Implementa la lógica de evaluación recursiva del árbol.
- **Clase Population:** Encargada de la inicialización de la población utilizando el método Ramped Half-and-Half (alternando entre árboles completos y en crecimiento).
- **Clase Evaluador:** Componente estadístico que calcula el Error Cuadrático Medio (MSE) comparando las salidas generadas por el árbol contra el vector objetivo.
- **Clase GeneticEngine:** Actúa como el controlador principal del algoritmo genético. Gestiona el ciclo de vida de las generaciones, implementando la selección por Torneo ($k=7$), el cruce de subárboles, las mutaciones (de punto y de subárbol) y el elitismo.
- **Clase DataLoader:** Módulo utilitario que extrae, limpia y formatea los archivos .csv y .data, asegurando que no existan valores nulos antes de iniciar la evolución.

Diagrama de Clases (UML)



Interfaz de la aplicación



Para facilitar la ejecución de los experimentos y el análisis visual, se desarrolló una interfaz gráfica web. En la parte superior, cuenta con un panel de Configuración que permite seleccionar el dataset a evaluar y ajustar fácilmente los hiperparámetros del algoritmo (como tamaño de población, generaciones y probabilidades). En la sección inferior, la pantalla se divide en dos áreas principales: a la izquierda se muestran los resultados del Mejor Individuo (Propio), incluyendo la fórmula matemática generada y el error final, mientras que a la derecha se integra una Terminal de Salida que permite monitorear el progreso del error del algoritmo generación tras generación en tiempo real

2. Desafíos de Implementación

Durante el desarrollo del motor genético, se presentaron retos técnicos significativos relacionados con la integridad de las operaciones y la manipulación de referencias en memoria:

Protección de Operadores (Closure): Durante la evaluación recursiva de fórmulas generadas aleatoriamente, es común encontrar operaciones matemáticas inválidas que interrumpen la ejecución. Para solucionar esto, se implementaron protecciones nativas con numpy dentro del método evaluar() de la clase Nodo:

- **División por cero:** Se utilizó np.where para evaluar si el divisor (en valor absoluto) es menor a $1e-10$; de ser así, la función retorna 1.0.
- **Logaritmo y Raíz Cuadrada:** Se aplicó np.abs antes de ejecutar np.log y np.sqrt para evitar el cálculo sobre números negativos. Además en el logaritmo se implementó un umbral mínimo de $1e-10$ para evitar el logaritmo de cero.

Cruce de Subárboles y Referencias en Memoria: Uno de los mayores desafíos fue implementar el Crossover de subárboles seleccionando nodos aleatorios sin perder la referencia a sus padres en el árbol original. Si simplemente se asignaba nodo_padre1 = nodo_padre2, Python creaba una nueva referencia local, perdiendo la conexión con la estructura principal. La solución fue intercambiar los atributos internos de los objetos (valor, izquierda, derecha) en lugar de los objetos mismos, garantizando una mutación estructural efectiva sin corromper el árbol.

3. Resultados y Comparativa (Implementación Propia vs gplearn)

Una vez que ya el motor se terminó de programar, se realizaron evaluaciones comparativas contra la librería gplearn utilizando los mismos hiperparámetros base (50 generaciones, población de 500 individuos, 90% de probabilidad de cruce y Torneo de 7).

Tabla de Resultados (MSE de Prueba)

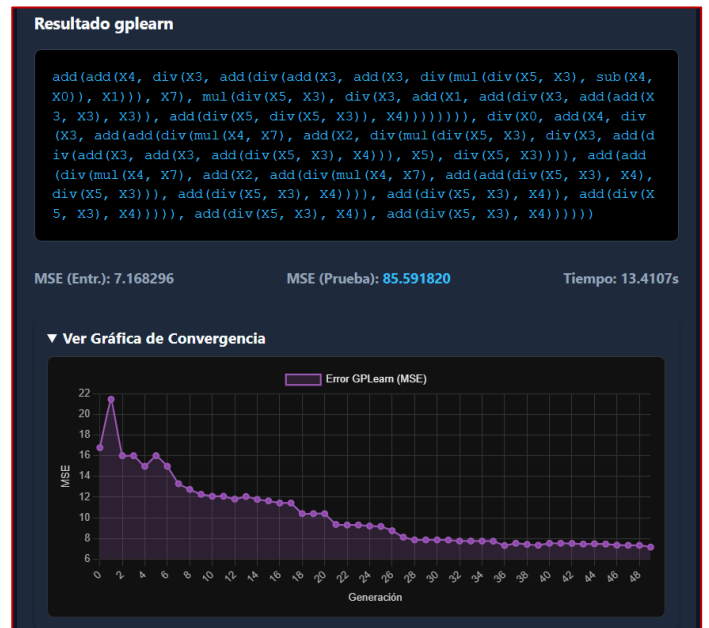
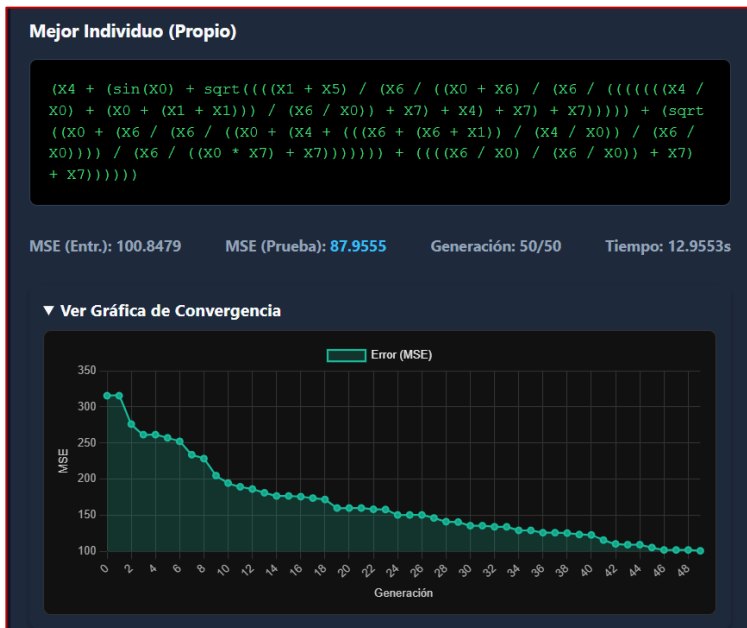
Nota: Los resultados muestran el Error Cuadrático Medio sobre el conjunto de prueba (30% invisible al entrenamiento) utilizando el dataset de Concrete.

Dataset	Implementación Propia (MSE)					gplearn (MSE)				
	G1	G2	G3	G4	G5	G1	G2	G3	G4	G5
Concrete	75.8131	89.9763	72.1434	88.2037	61.1554	59.7692	110.6863	109.4042	181.5383	125.0289
Housing	20.7707	22.4492	23.4315	25.8115	25.1351	22.1363	25.9882	45.4290	18.0729	35.8671
Yacht	2.4177	2.0093	7.5993	1.8555	1.7597	6.9058	12.5497	6.6257	4.3131	6.4567
Tower	15.1159	12.0599	20.0743	19.7078	14.7617	72.8868	78.5445	55.8973	46.0346	14.0064
Energy Cooling	11.8976	11.3880	5.9675	5.9235	13.1878	13.1484	12.9749	17.4480	12.9381	13.2689
Energy Heating	7.1380	10.4258	5.4971	10.5942	3.0163	9.6249	18.1359	15.3794	8.4230	14.4061

Dataset	Implementación Propia		gplearn	
	Media	Desviación Estándar	Media	Desviación Estándar
Concrete	77.45	11.92	117.28	43.59
Housing	23.52	2.03	29.50	11.08
Yacht	3.12	2.51	7.37	3.07
Tower	16.34	3.45	53.47	25.61
Energy Cooling	9.67	3.47	13.95	1.96
Energy Heating	7.33	3.25	13.19	4.07

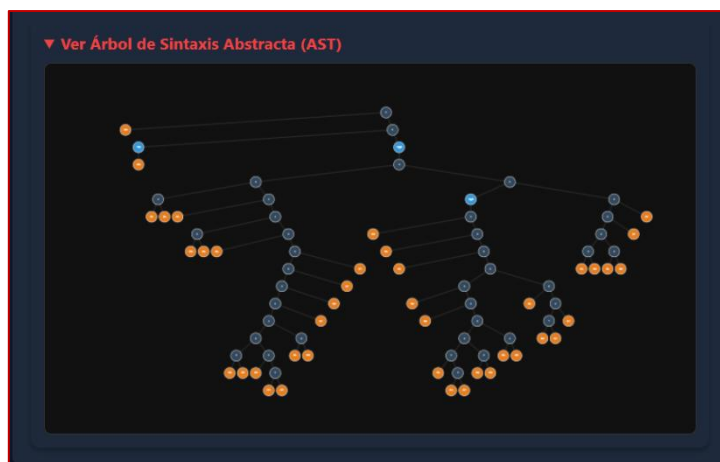
Gráficas de Convergencia

A continuación, se observa el comportamiento del error a lo largo de las generaciones para el dataset Concrete:

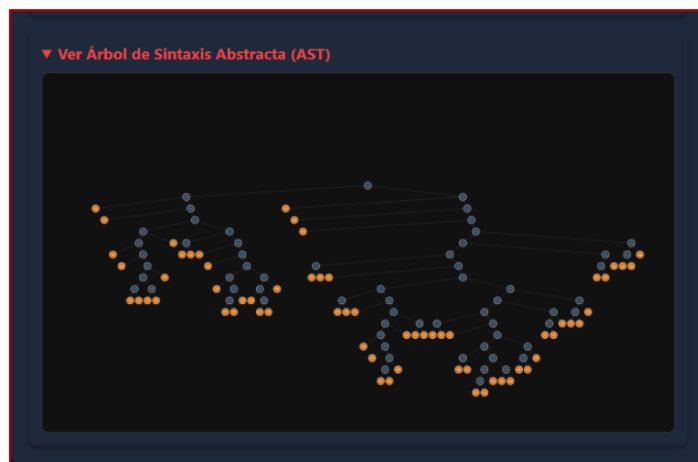


Árboles de Sintaxis Abstracta

A continuación, se muestran capturas de los árboles correspondientes tanto de nuestro motor como el de gplearn:



Nuestro motor



Motor de gplearn

Análisis de Velocidad

Al medir los tiempos de ejecución total para 50 generaciones, se observó algo interesante. Teóricamente la librería gplearn posee una ventaja arquitectónica masiva al realizar evaluaciones vectorizadas optimizadas en C (mediante Cython), mientras que nuestra implementación depende de una evaluación recursiva en Python puro, lo cual suele ser penalizado por el costo computacional del Call Stack.

Sin embargo, en las pruebas con el dataset Concrete, nuestra implementación finalizó en 12.9553 segundos, superando ligeramente a gplearn, que tomó 13.4107 segundos. Este resultado se explica al observar la complejidad de los árboles generados: gplearn sufrió de un crecimiento excesivo (Bloat) tan severo que la inmensa cantidad de nodos a evaluar neutralizó su ventaja de ejecución en C. Y en nuestra implementación aprovechó de manera eficiente las operaciones de numpy en las hojas del árbol, logrando un tiempo altamente competitivo en estructuras menos colosales.

Complejidad de la Solución y Fenómeno del Bloat

Al observar las imágenes de los árboles finales, se nota claramente el impacto del fenómeno del Bloat en ambas implementaciones. En el árbol de nuestro programa, la estructura creció de forma masiva tanto a lo ancho como a lo profundo, creando una red inmensa de operaciones matemáticas. Esto ocurre porque al usar una tasa de cruce muy alta (90%), el algoritmo hereda ramas enteras de los padres que muchas veces no aportan valor real a la predicción final.

Por otro lado, el árbol de gplearn también resultó ser una estructura gigante y sumamente compleja. Aunque gplearn incluye métodos internos más avanzados para intentar penalizar o controlar el crecimiento de las expresiones, la realidad es que para el dataset de Concrete, ambos algoritmos terminaron generando redes inmensas de nodos que son imposibles de rastrear o analizar visualmente de principio a fin.

Calidad de la Fórmula

Siguiendo con el dataset de Concrete, al comparar las fórmulas que obtuvimos, nos damos cuenta de que aunque funcionan para predecir, son difíciles de leer. En las fotos de los árboles se nota por qué: el árbol de nuestro código es una red gigante de nodos que se extiende por todos lados. La fórmula resultante mezcla sumas, raíces y senos de una forma tan enredada de la cual sería difícil sacar una conclusión clara sobre cómo los materiales afectan la resistencia.

Por su parte, la fórmula de gplearn no es mucho mejor. Aunque el árbol se ve un poco más distribuido en el espacio, la expresión matemática está llena de funciones `add()`, `div()` y `mul()` repetidas una y otra vez. Esto nos confirma que el algoritmo se dedicó a "memorizar" el ruido de los datos en lugar de encontrar una relación física sencilla. Al final, ninguna de las dos herramientas nos dio una fórmula que se pueda usar fácilmente en un manual técnico, lo que demuestra que la programación genética tiende a crear soluciones muy complejas si no se controla el crecimiento del árbol.

Conclusión

Hacer este proyecto nos sirvió para entender a fondo cómo funciona la Programación Genética y cómo las computadoras pueden evolucionar soluciones matemáticas. Fue un reto importante programar desde cero la evaluación de los árboles y sobre todo, las protecciones para que el código no tuviera errores como las divisiones entre cero o los logaritmos negativos.

Lo más interesante de la comparativa fue ver que, aunque nuestro motor es más sencillo y corre en Python puro, logró resultados muy parecidos a los de una librería profesional como gplearn, e incluso le ganamos en tiempo en algunas pruebas porque sus árboles crecieron de más. Al final aprendimos que la clave de estos algoritmos no es solo que encuentren una respuesta con poco error, sino lograr que esa respuesta sea lo suficientemente simple para que un humano la pueda entender.